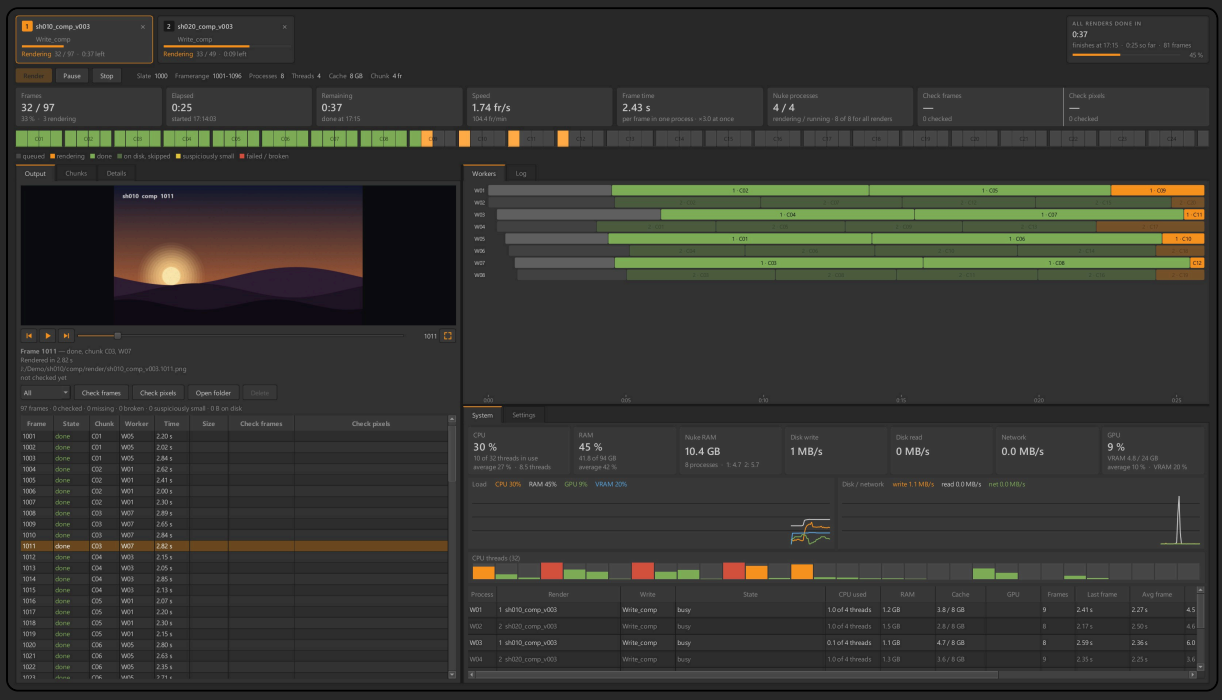
USER GUIDE · VERSION 0.99.6

JKrender

Fast local rendering for Nuke. A render is split into pieces (chunks) that several Nuke processes render at once. After the render JKrender checks every frame on disk.



Contents

1 How JKrender works

2 Installing and starting from Nuke

3 Map of the window

4 Render cards and the control row

5 Tiles and the frame map

6 The Output panel: preview and frames

7 The Chunks and Details panels

8 The Workers and Log panels

9 The System panel

10 Settings: every setting

11 How to set up a render

12 Test this script

13 Watching RAM and VRAM

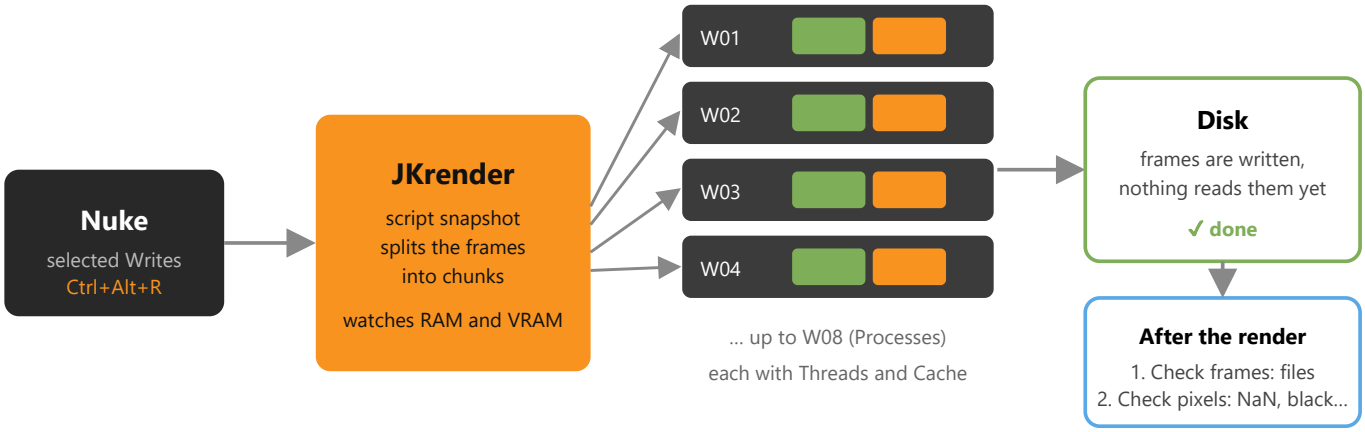
14 Checks, sounds, crashes, data

15 Mouse and keys: cheat sheet

16 When something does not work

1 How JKrender works

A normal Nuke render runs in one process that computes the whole script frame by frame. One Nuke rarely uses all 32 threads of the CPU, though. So JKrender starts **several Nuke processes side by side**. Each **opens the script only once** and then gets pieces of work, called **chunks**: for example 10 frames in a row.



Chunks get smaller towards the end, so all processes finish at about the same time.

Three numbers that decide the speed

Processes

How many Nuke processes run at once (for all renders together). More processes do more work at once, but also take more RAM and VRAM.

Threads

How many CPU threads each process gets (Nuke `-m`). **Processes × Threads** should come to about the thread count of the PC (32 here).

Cache

The cache limit of one process in GB (Nuke `-c`). A process really takes more. How much, System › RAM shows.

> The default is **8 processes × 4 threads**. Heavy comp scripts that one Nuke already fills (Defocus, Denoise) are often faster on **4 × 8** and take half the RAM. Scripts whose VRAM grows want fewer processes. What is best for your script, **Test this script** measures (chapter 12).

Where things are kept

What	Where
Settings	`~/.nuke/JKrender/settings/settings.json` (every user has their own). Older versions kept them in `~/.jkrender`; JKrender moves them on its first start.
Render logs	`~/.nuke/JKrender/log/`: one folder per render (script snapshot, job, the log of every Nuke process and `report.json`). Older than 15 days are deleted.
Test results	`~/.nuke/JKrender/settings/test_results`
Rendered frames	Where the Write sends them. JKrender never deletes them by itself, only with the Delete button.

2 Installing and starting from Nuke

Install (once)

1 Copy the whole `JKrender` folder into `%USERPROFILE%\.nuke\`. Downloaded from GitHub as a zip, it is called `JKrender-main`: rename it to `JKrender`.

2 Add this line to `~/.nuke/init.py` (create the file if it is not there):

```
nuke.pluginAddPath("./JKrender")
```

3 Restart Nuke. On the first start, JKrender asks to download numpy and OpenEXR for the EXR preview (about 15 MB). If you say no, rendering still works, only without the EXR preview. To be asked again, delete the file `lib/DECLINED`.

Optional: for details and the player of MOV/MP4, put ffprobe/ffmpeg on PATH (`winget install ffmpeg`). Nuke 16 is needed (PySide6).

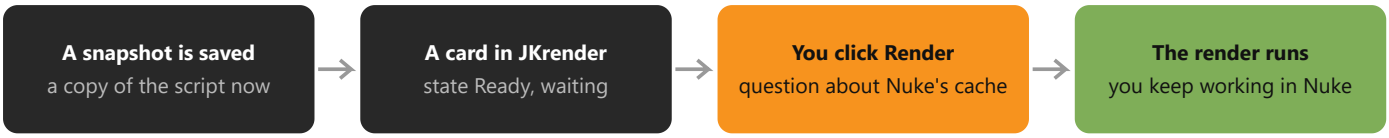
A new version: copy the new files over the old ones. The `lib`, `log` and `settings` folders stay.

The JKrender menu in Nuke

Item	What it does
Render Selected Write(s) `Ctrl+Alt+R`	Sends the selected Writes (or Groups with Writes) to JKrender. Every Write gets its own card. If the window is not open yet, it opens.
Open Renders Folder	Opens the folder with the render logs.

If the script is not saved, it asks *Save it and render?*: the saved version is rendered.

What happens when you send it

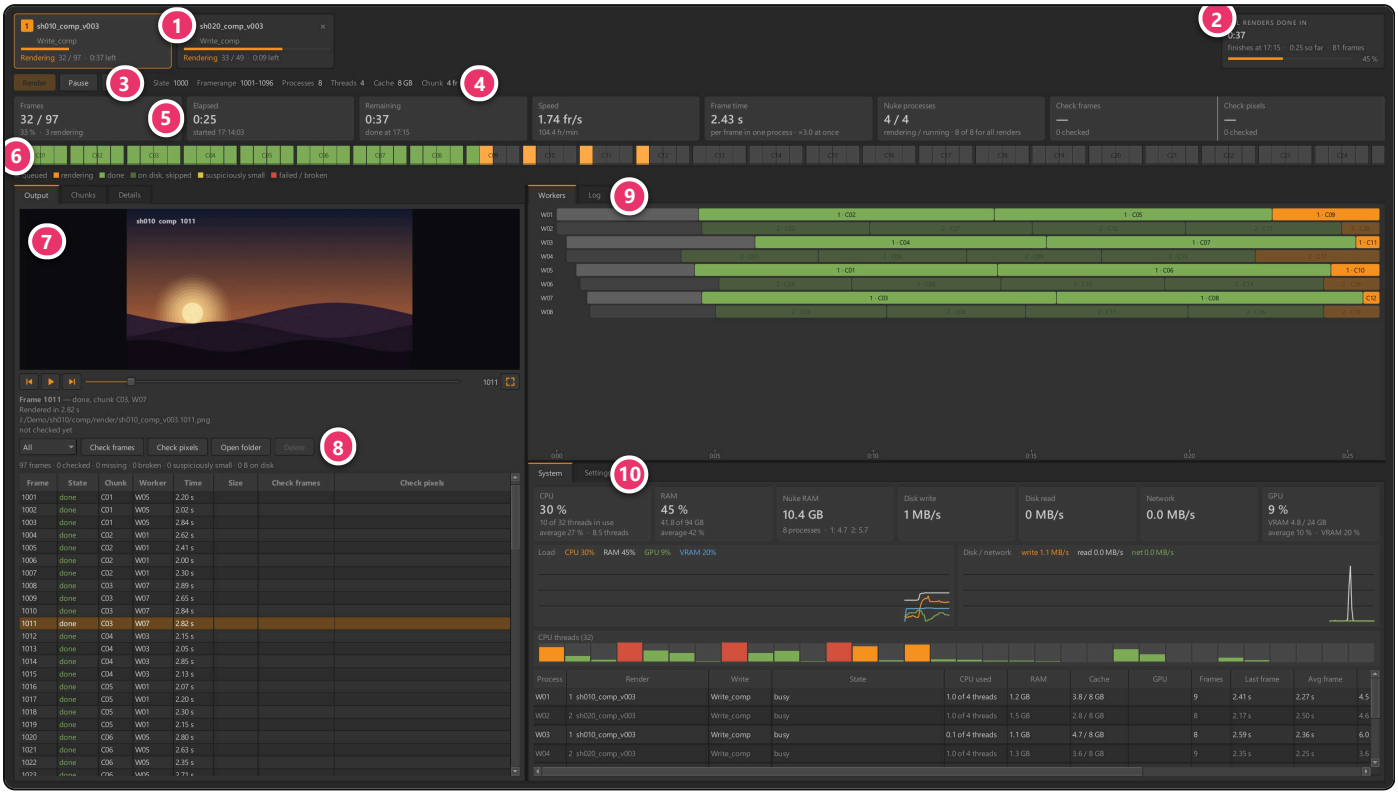


Snapshot: the script is rendered as it was when you sent it. You can keep working and saving in Nuke straight away; the running render is not affected. To render a changed script, send it from Nuke again: the newer send replaces the old card.

Cache question: when a Nuke is open, Render asks whether to clear its buffers, playback cache and Blink cache, so the render has more memory. The default answer is **No**, and the render starts either way. The disk cache is not touched. After updating JKrender, restart Nuke once for this to work.

3 Map of the window

At the top are the render cards and the control row, under them the numbers and the frame map. The rest is three panels: **Output / Chunks / Details** on the left, **Workers / Log** top right and **System / Settings** bottom right. The borders between the panels can be dragged.



1. Render cards: one Write (render) per card. A click picks it.
2. When all renders should be done.
3. Render, Pause, Stop for the picked render.
4. Slate, Framerange, Processes, Threads, Cache and Chunk of this render (to read).
5. Tiles with numbers: frames, time, speed, processes, checks.
6. The frame map by chunks. A click on a frame shows it.
7. Output: preview of a frame with playback.
8. Output: filter, checks, folder, delete and the list of frames.
9. Workers: a timeline of every Nuke process.
10. System: load of the PC and the table of processes.

4 Render cards and the control row

A render card

Every Write sent is one card: number, script name, Write name, a progress bar and the state (Ready, Waiting, Rendering, Paused, Checking output, Done, Failed, Stopped). A **click** picks the card and the whole window then shows that render. **Shift+click** picks several cards; Render, Pause and Stop then act on all of them, and Render again on finished ones too (it asks once for all). The **×** closes the render; the frames on disk stay. **A red card**: the render has an error (it failed, Nuke did not render a frame, broken frames, a Nuke error, a process crashed), also while it renders; the card's tooltip says which. A process closed for RAM or VRAM is not an error.

Renders share one budget of processes. At most as many run side by side as Settings *Max renders at once* says (default 3); the processes are shared between them (8 → 4 + 4 → 3 + 3 + 2).

The card on the right: ALL RENDERS DONE IN

When all renders that run or wait should be done, at the speed the PC renders now. Under it: when they end, how long they have run and how many frames are left. When all are through, the card shows **ALL RENDERS TOOK**: how long it all took.

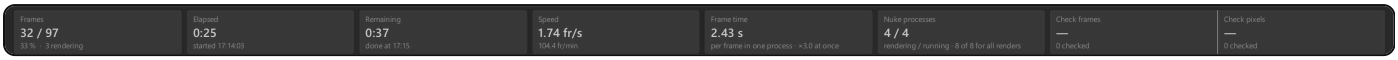
The control row

Element	What it does
Render	Starts the picked render (or several picked). Frames already fine on disk are skipped. After Stop the render goes on where it ended (Resume). When the render has frames on disk (finished, stopped, or with frames from before), it asks: **Only missing** renders only the frames missing or broken on disk (the others are dark green in the map), **All frames again** renders everything again, **Cancel** nothing.
Pause	Freezes all Nuke processes of the render mid-frame (the CPU is free, the memory stays held). Another click goes on.
Stop	Stops the render, ends its processes and deletes half-written frames and Nuke's temp files. Finished frames stay.
Slate	The slate's number: the frame before the Framerange, when the slate is on (Settings › Job).
Framerange	The frames of this render: the range from Nuke, unless you changed it in Settings › Job.
Processes · Threads · Cache · Chunk	What the render runs with. Changed in Settings › Processes and Chunks; the number of processes also while it runs, with a right click in System.
Coloured bar	The result of the render, see below.

The coloured bar with the result

Colour	What it means	Example
Green	Done, all frames are on disk and pass the check.	Done in 2:06. All 97 frames are on disk and pass the check.
Yellow	Done, but something is not verified or looks suspicious (frames not checked, suspiciously small; a slate is not reported as small).	frames not checked · 2 suspiciously small
Red	An error: the render failed, broken frames, or Check pixels found a problem (NaN, a black frame…).	2 broken frames · Output: Problems shows them
Grey	In progress: *Checking pixels…*, waiting for free processes and so on.	Waiting: 1 render already uses the PC

5 Tiles and the frame map



Tile	What it shows
Frames	Done / all frames of the render, per cent and how many are rendering now.
Elapsed	How long the render has run (without pauses).
Remaining	The time left at the current speed.
Speed	Frames per second and per minute, all processes together. The best number to compare settings.
Frame time	How long one frame takes in one process, and how many frames are rendered at once (× N at once).
Nuke processes	How many processes render / run, plus those loading or waiting.
Check frames	The file check: missing, empty, cut-off, unreadable and suspiciously small frames.
Check pixels	The pixel check (EXR, PNG, JPEG, TIFF, DPX): black, empty and one-colour frames, frames the same as another, and a frame whose size, channels or bit depth differ from the rest. In EXR also NaN, inf and compression.

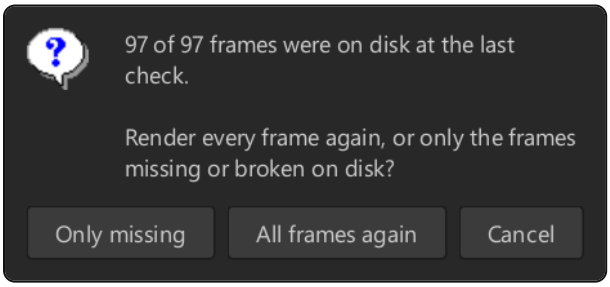
The frame map



Each little box is one frame, grouped by chunks (C01, C02…). A **click on a frame** opens it in the preview. Colours:

waiting (queued) rendering done already on disk, skipped (Only missing) suspiciously small failed / broken

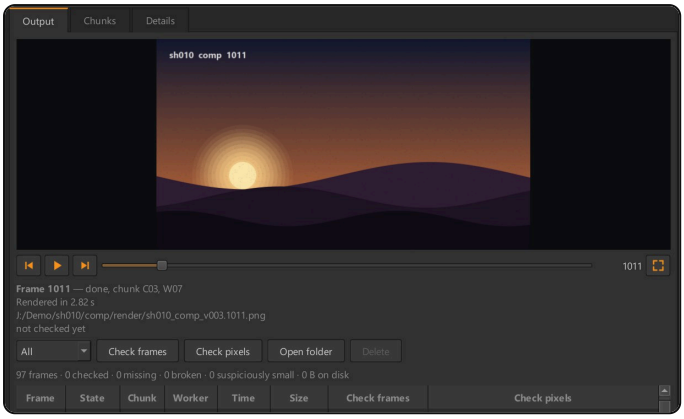
Render again: everything, or only what is missing



When the render has frames on disk and you click **Render again**, it asks: **Only missing** (the default) renders only the frames missing or broken; the others are skipped and **dark green** in the map. **All frames again** renders everything again, **Cancel** nothing. A movie (MOV, MP4) does not ask: a movie is one file and is rendered whole.

With several cards picked with **Shift** it asks once for all (e.g. *27 of 27 frames of 3 renders*…). Frames that were on disk when the render opened, or from the run before Stop, are dark green. In Output the filter **Rendered** then shows only the frames of this run, **All** all of them.

6 The Output panel: preview and frames



Preview

- **Mouse wheel**: zoom, **dragging** moves the image.
- **Double click**: back to the whole frame. When it already is, the preview grows over all three panels; back with `Esc`.
- Zoomed in, the full resolution is loaded (EXR and 4K too).
- `Space` starts and stops playback. It never presses a button, so it never stops the render.

The bar under the preview

Previous frame · **Play/Pause** (plays the finished frames in a loop at the script's fps) · next frame · a slider over all frames · frame number · **enlarge** (preview over three panels, `Esc` back). Under the bar: the frame's path, size, resolution, channels and check results.

Buttons

Element	What it does
Filter	**All**: every frame at once: rendered, skipped, broken, *not on disk* and those still to render, and frames on disk outside the Framerange. The other filters are parts of it: **Rendered**: only frames rendered in this run. **Problems**: errors, suspiciously small frames and what the pixel check found. **Framerange**: only the frames of the chosen range.
Check frames	Checks the files of all frames again: there, not empty, whole and readable, and not far smaller than the rest. A frame not on disk yet stays *not on disk* and does not count as broken.
Check pixels	Checks the pixels of every frame in the background (EXR, PNG, JPEG, TIFF, DPX): black, empty and one-colour frames, frames the same as another anywhere in the sequence, a different size, channels or bit depth; in EXR also NaN, inf and compression. About 0.1 s per 4K frame. It only reports, nothing renders again.
Open folder	Opens the output folder.
Delete	Deletes rendered frames from disk: all of them (the default) or a range you give. It asks first.

The list of frames

Columns: **Frame**, **State**, **Chunk**, **Worker** (which process rendered it), **Time**, **Size**, **Check frames**, **Check pixels**. With the All filter every frame is listed from the start and its state changes as it renders. After **Render again** frames do not disappear: they stay as *on disk, queued again* with their old size and time (you can view and play them) until the new render overwrites them. Frames on disk outside the chosen Framerange stay in the list, marked *not in Framerange*.

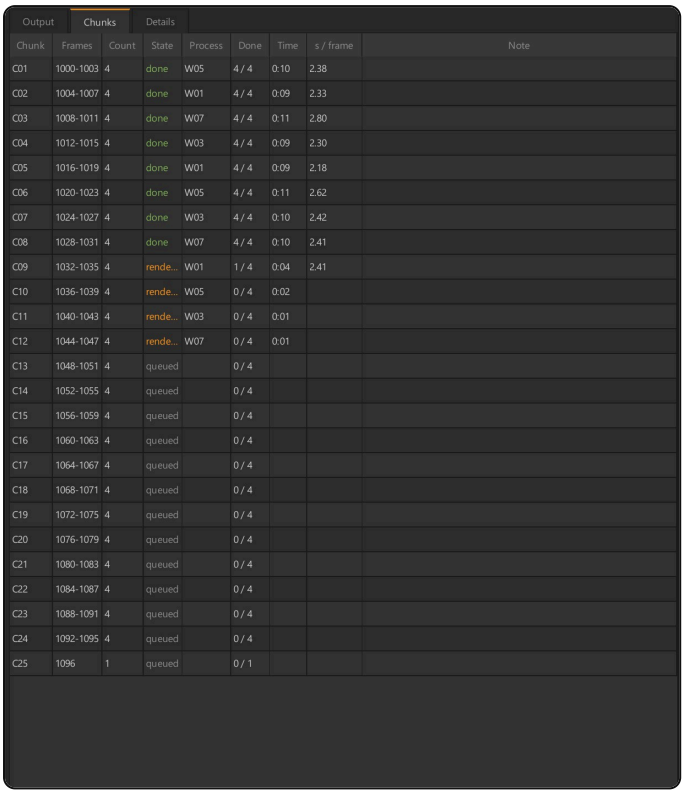


Right click on a frame (or on several selected):
Render this frame / batch: renders only these frames again (while the render runs, they are added to it).
Delete this frame / batch: deletes them from disk (only while the render does not run).

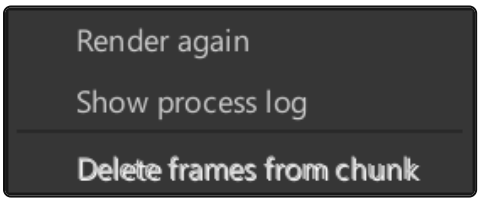
> **MOV / MP4 / MXF:** instead of the list of frames there is one movie file with its details (codec, profile, fps, resolution, timecode, frame count against the Write's settings) and a player with zoom. Buttons Check video, Open folder, Delete video. A movie cannot be split, so one process with every thread renders it; Settings show it so too (Processes 1, locked). The settings of the other renders do not change.

7 The Chunks and Details panels

Chunks



One row per chunk: **Chunk** (C01…), **Frames** (range), **Count**, **State**, **Process** (which renders it), **Done**, **Time**, **s / frame** and **Note** (why a chunk was rendered again and so on).



Right click: Render again (the chunk again), **Show process log** (the log of the process that rendered it), **Delete frames from chunk** (only while the render does not run).

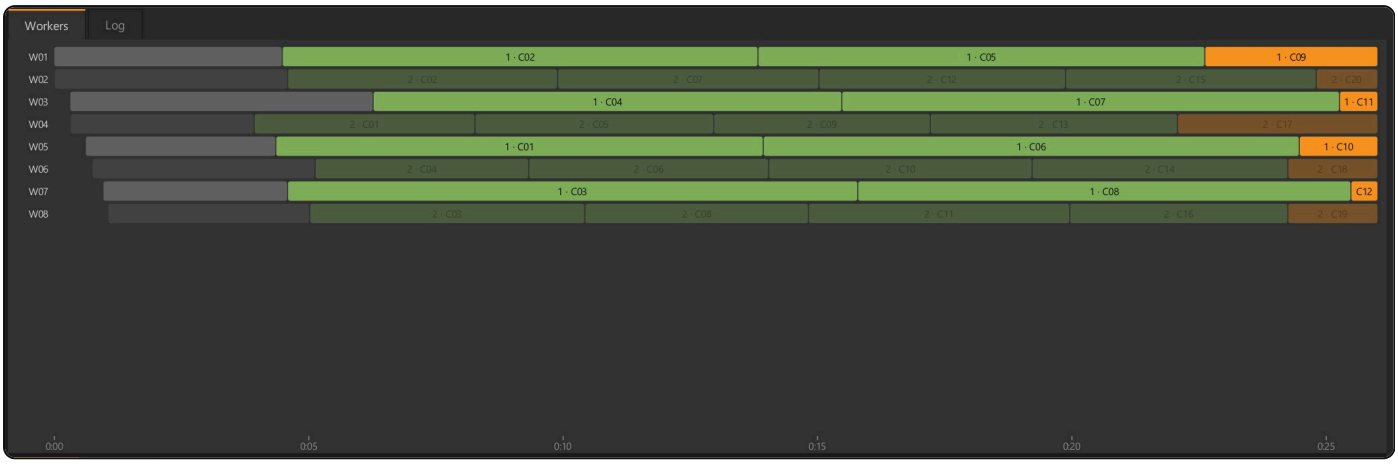
Details



Everything about the render in one place: the script and its snapshot, the Writes and output paths, format, fps, range, Nuke version, the Processes × Threads × Cache it runs with, times and the paths to the logs. Handy when you need to know exactly what was rendered.

8 The Workers and Log panels

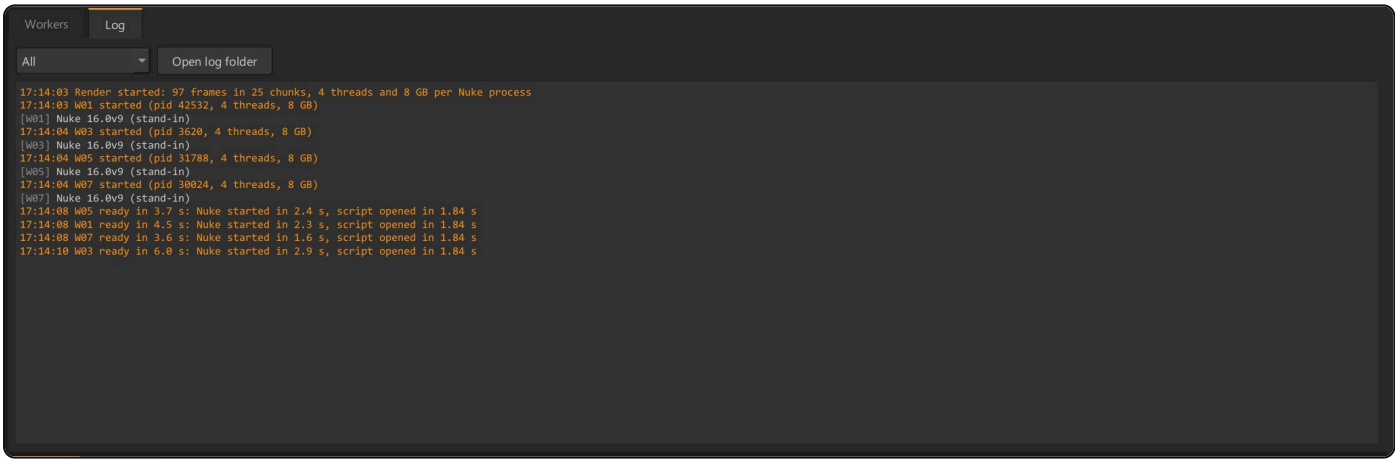
Workers: a timeline of the processes



One row per Nuke process (W01–W08), time from left to right. ⬛ **grey**: Nuke starts and opens the script (also when it is swapped for a fresh Nuke, labelled *again: reload*). 🟩 **green**: finished chunks (with the chunk number). 🟧 **orange**: the chunk rendering now. The chunks of other renders are dimmed (Settings › *Dim other renders in Workers*). You see at once whether a process stands still, starts again or takes long to open the script.

A right click on a process row opens the same menu as in the System panel (chapter 9).

Log



JKrender's messages and the output of every Nuke process, with the time. Filter: **All**, **JKrender only** (only JKrender's messages), **Errors and warnings**. Open log folder opens the folder with this render's logs: the script snapshot, `job.json` and the `worker_NN.log` of every process.

> Every process reports in the log how long it took to be ready, split into **Nuke starting** and **opening the script**. If starting takes 15 s or more, JKrender warns. A slow start is usually the licence or the pipeline's start-up scripts, not the script itself.

9 The System panel



Tiles and graphs

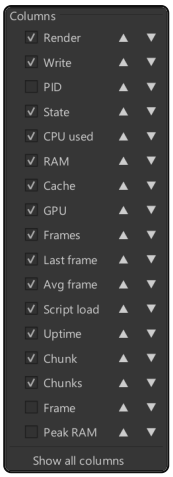
CPU (use and threads), **RAM** (total), **Nuke RAM** (what the Nuke processes take), **Disk write**, **Disk read**, **Network**, **GPU** (use and VRAM). The **Load** graph shows CPU, RAM, GPU and VRAM in %, the **Disk / network** graph transfers in MB/s. **CPU threads**: one bar per CPU thread, so you see whether the whole PC is busy.

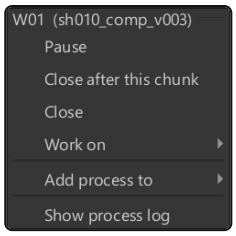
The table of processes

Column	Meaning
Process	Process number (W01…).
Render · Write	Which render it works on, and its Write's name.
State	What it does: loading, busy, idle, paused, closing… *busy → reload (RAM 91 %)*: after its chunk it gets a fresh Nuke, because the RAM is near the limit (chapter 13). What it waits for, e.g. *wait: VRAM 75 → 14 % (3 starting)*: now 75 %, one more process fits under 14 %, because 3 processes are still starting and JKrender keeps room for them; the tooltip says more.
CPU used	How many CPU threads the process really uses.
RAM · Cache	The real memory of the process and its Nuke cache against the limit.
GPU	Graphics card use and VRAM of this process.
Frames · Last frame · Avg frame	How many frames it rendered, how long the last took and the average.
Script load	How long Nuke took to start and open the script.
Uptime · Chunk · Chunks	How long it has run, which chunk it is on and how many it did.
PID · Frame · Peak RAM	Hidden columns: the Windows process ID, the current frame, the highest RAM.

A click on the table header opens the list of columns: tick to show or hide a column, the arrows change the order. Columns can also be dragged by the header. Widths and order are saved when the window closes.

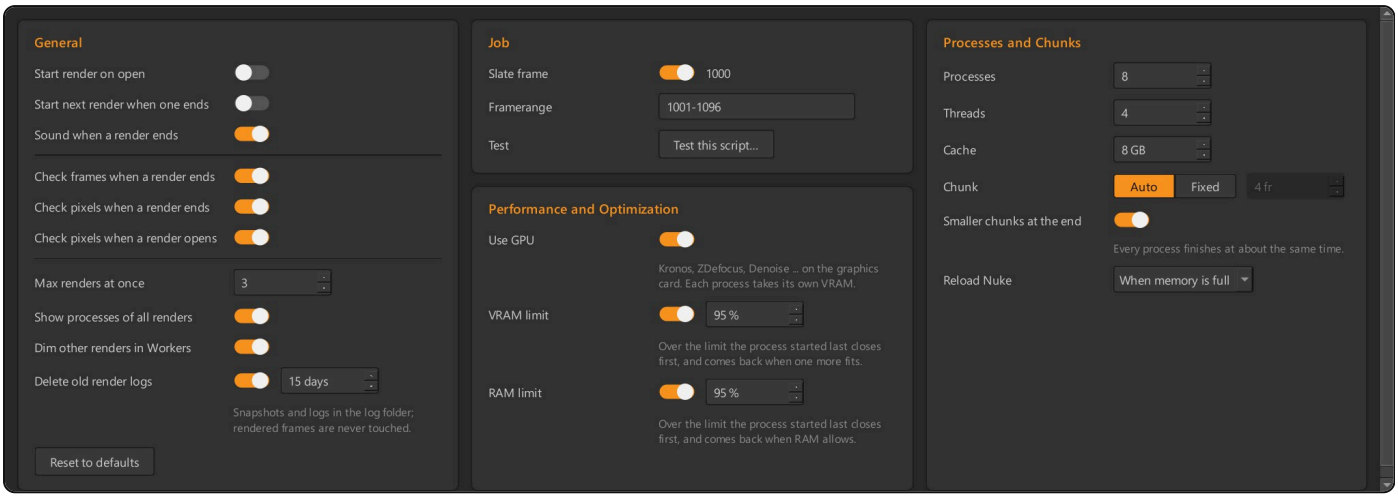
A right click on a process (in the table or on a row in Workers) opens the menu on the right. With several renders running, *Add process to* has a submenu to pick the render.





Menu item	What it does
Pause / Resume	Stops or lets go only this process.
Close after this chunk	The process finishes its chunk and closes, also a paused one or one already closing for memory. **Processes** goes down by 1 and stays so.
Close	Closes the process now. Its chunk goes back to the queue. Another may then take its place.
Work on › render	Moves the process to another running render.
Add process	Starts one more process (also with a click on an empty spot of the table). **Processes** goes up by 1 and stays so. The RAM and VRAM limits still apply.
Start again · Remove process	For a closed process: starts it again, or removes it (also an empty place where no process runs). A removed one does not come back and **Processes** goes down by 1.
Show process log	Opens the log of this process.

10 Settings: every setting



Settings are saved at once. Each has a tooltip. **Reset to defaults** puts everything back as JKrender comes, the table columns too; what JKrender learned about scripts stays.

General

Setting	What it does	Default
Start render on open	The render starts as soon as the window opens, without clicking Render.	off
Start next render when one ends	When a render ends or fails, the next render not started yet, in the order of the cards, starts by itself. A render you stopped starts nothing.	off
Sound when a render ends	A sound when a render ends, once the checks that are on are through. Another sound on an error, broken frames or a problem in the pixels. A stopped render is silent.	on
Check frames when a render ends	When the render ends (also after Stop), every rendered frame once: there, not empty, whole and readable, not far smaller than the rest. Nothing reads the output folder while it renders. Off: a broken frame counts as done until you press Check frames.	on
Check pixels when a render ends	Right after the frame check, the pixels of all frames in the background (black, one-colour, the same, a different format; in EXR also NaN and inf). It only reports.	on
Check pixels when a render opens	The same check of the frames already on disk, as soon as a render opens. Render stops it.	on
Max renders at once	**1**: renders go one after another; the next starts loading while the last one finishes its last chunks (the first shot is done first). **2–3**: they run side by side and share the processes (not faster overall, but you see frames of every shot early).	3
Show processes of all renders	Workers and System show the processes of all renders. Off: only of the picked one.	on
Dim other renders in Workers	On the timeline the chunks of the picked render are bright, the others dimmed.	on
Delete old render logs	When JKrender opens, deletes log folders older than the days set. Never frames.	on, 15 days

Thin lines separate the groups in General. Rendering again, the Workers timeline always shows only the new run.

Job (for the render picked at the top)

Setting	What it does	Default

Slate frame	On: every render also renders the frame before its Framerange as the slate (1001-1020 → slate 1000, 1050-1060 → 1049). Next to the switch: the slate number of the picked render.	on
Framerange	The frames of the render, e.g. `1001-1100` or `1001-1010,1050`. Every render has its own. An empty field brings back the range from Nuke. Confirm with Enter or by clicking elsewhere. Locked while it renders. Frames on disk stay.	from Nuke
Test › Test this script…	Measures settings for this PC on frames from the whole shot and shows the fastest (chapter 12).	—

Performance and Optimization

Setting	What it does	Default
Use GPU	Starts Nuke with `--gpu`, so GPU nodes (Kronos, ZDefocus, Denoise…) render on the graphics card. A light 4K script with Kronos was 3× faster. Every process takes its own VRAM.	on
VRAM limit	Over the limit fewer processes run. At 97 % the process started last closes at once; also when the card is full and no frame finishes for a long time. Within 5 % of the limit Reload Nuke helps. Chapter 13.	on, 95 %
RAM limit	Over the limit fewer processes run: the one started last closes first. When memory is free again, it comes back. So Windows never starts swapping.	on, 95 %

Processes and Chunks

Setting	What it does	Default
Processes	Nuke processes for all renders together.	8
Threads	Threads per process (`-m`).	4
Cache	Nuke's cache limit per process in GB (`-c`). What a process really takes, System › RAM shows. A cache too small slows the render down: Nuke then throws away and computes again.	by RAM
Chunk: Auto / Fixed	**Auto**: about 4 chunks per process, at most 10 frames (46 frames with 8 processes → 2, 96 → 3, 320+ → 10). **Fixed**: the size you set on the right. Applies to chunks not started yet.	Auto
Smaller chunks at the end	Towards the end of a render, chunks get smaller down to single frames, so all processes finish at about the same time.	on
Reload Nuke	After its chunk a process closes Nuke and opens the script in a new one, with its RAM and VRAM back at the start. It costs one script load. **When memory is full**: only when the RAM or VRAM has stayed within 5 % of its limit for 30 s, one process at a time. **After every chunk**: after every chunk. **Never**: never. Chapter 13.	When memory is full

> Processes, Threads and Cache are locked while a render runs. For a movie they show 1 process with every thread. To change the number of processes while it renders, right-click in System (*Add process*, *Close after this chunk*, *Remove process*).

11 How to set up a render

The recommended way for a new or heavy script:

1 **Send the Write** Ctrl+Alt+R

2 **Create tests** setups for this PC

3 **Start the test** Round 1 and Round 2

4 **Use these settings** takes the green row

5 **Render** and watch System

Steps 2–4 are in the Test this script window. Light scripts can go straight on the default 8 × 4; a test pays off when a render takes tens of minutes.

A guide (Ryzen 9950X, 32 threads, 94 GB RAM, RTX 3090 24 GB)

Kind of script	Processes × Threads	Why	Measured
Light comp, keying, grade	**8 × 4**	One Nuke does not use all threads, more processes do.	a 4K test script: 8 × 4 the fastest
Heavy comp (Defocus, Denoise, many layers)	**4 × 8** or **2 × 16**	One process uses more threads. 8 processes run out of RAM and processes close.	heavy comp: 4 × 8 with 12 GB cache 17 fr/min, 2 × 16 16 fr/min
A script whose VRAM grows (GPU nodes)	**1–2 × 32/16**	VRAM grows with every frame; more processes do not fit on the card.	VRAM +1.7 GB per frame → 1 × 32

How to tell the settings are not good

Short of memory

System › RAM is at the limit, State says *wait: RAM …*, processes close and start again (grey blocks in Workers). **Fix:** fewer processes with more threads, or a smaller Cache.

A full graphics card

VRAM at 95 %, processes close for VRAM. JKrender itself suggests fewer processes with more threads (it changes nothing by itself). **Fix:** fewer processes with more threads (Test this script shows how many), or turn Use GPU off.

CPU not used

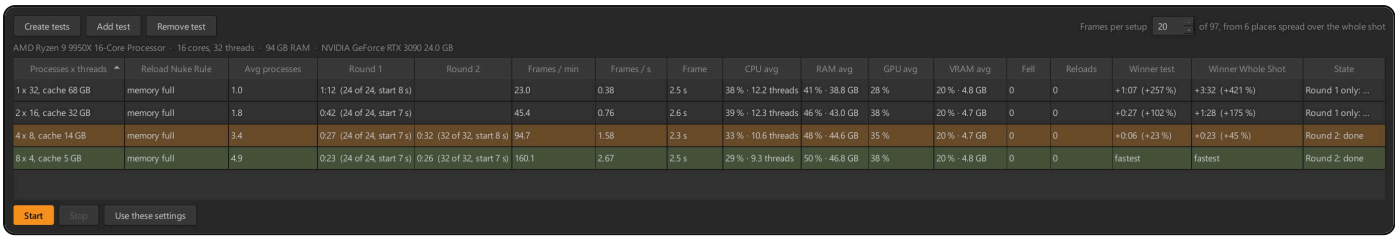
CPU threads are mostly empty and RAM has room. **Fix:** add a process (right click › Add process) or raise Threads.

Long script opening

Grey blocks in Workers are long, Script load is high. At start the Nukes compete for licences and start-up scripts. **Fix:** longer chunks hide it; it usually has nothing to do with the script itself.

> **The best measure is the Speed tile** (frames per minute). Compare it between settings on the same script, or leave it to *Test this script*.

12 Test this script



A finished test: the green row is the fastest, the orange one second. Winner test shows how much longer the test took with the others, Winner Whole Shot the same for the whole shot.

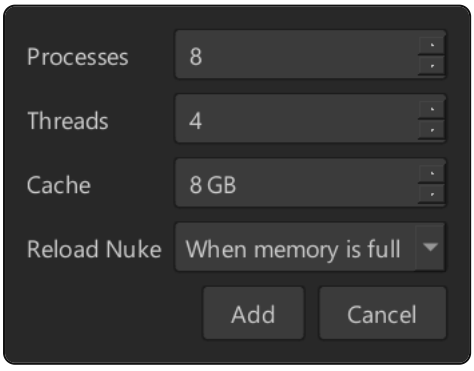
Buttons at the top

Create tests looks at the computer (the line under the buttons: CPU, cores and threads, on Intel performance and efficiency cores, RAM, graphics) and makes **7 setups** from 1 process with all threads to as many processes as the PC takes: each process at least 4 threads and at least 4 GB cache. On 32 threads and 94 GB: 1×32, 2×16, 3×10, 4×8, 5×6, 6×5, 8×4. Past 64 threads a process gets at most 64 (one processor group in Windows). It replaces everything in the table.

Add test adds a setup of your own (Processes, Threads, Cache and Reload Nuke), Remove test removes the selected ones. **Right click** on a row: **Edit test**. Your own tests are kept. The Reload Nuke rule of each setup is in the **Reload Nuke Rule** column right after Processes x threads: *never*, *memory full* or *every chunk*.

The **Cache** of each setup is all the RAM for Nuke (the RAM limit, less a reserve for Windows) divided by the number of processes, less the 4 GB a process takes outside its cache. What runs at the moment does not matter.

Frames per setup (top right): how many frames each setup renders in Round 2. When the shot has fewer frames, it says so and renders all of them.



How the test runs

Frames are taken from **6 places spread over the whole shot** (from the first frame to the last), a few frames in a row at each. **All setups of a round render the same frames.**

Round 1: every setup briefly, 3 frames per process of the setup with the most processes (on 32 threads 24 frames), at most 3 minutes. **Round 2:** the two fastest (and any within 5 % of the best) again with exactly Frames per setup frames, but at least 4 per process (6 processes → 24).

Winner test: how much longer the test took than with the fastest (the same frames, Nuke's start included). **Winner Whole Shot**: an estimate for the whole shot = the test's time + the remaining frames at Frames / min. The start is paid once, the slower rate on every frame, so on a long shot the difference grows (e.g. test +0:10, whole shot +0:43). The cell's tooltip has both times.

Frames / min is counted once the processes are going: Nuke's start and the first frame of each process do not count; in a long render they happen only once. **Frames / s** is the same per second.

Columns: **Reload Nuke Rule**, **Avg processes** (how many processes rendered on average), **Round 1**, **Round 2** (time of the round, frames, start), **Frames / min**, **Frames / s**, **Frame**, **CPU / RAM / GPU / VRAM avg**, **Fell** (processes that fell), **Reloads** (how often Nuke was reloaded), **Winner test**, **Winner Whole Shot**, **State**. A click on a header sorts; columns can be resized and moved.

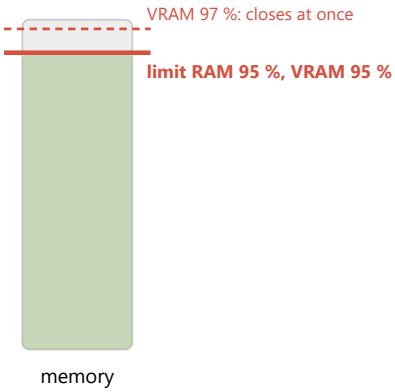
Use these settings takes the green row into Settings, Reload Nuke too. Frames go to a test folder and are deleted; the Write's render hooks do not run in a test. Tables stay in `~/.nuke/JKrender/settings/test_results`.

The **Add test** window (and the same **Edit test** from the right click).

> Nothing else should render during a test: it would measure two renders fighting over the PC. The test does not start next to a render.

13 Watching RAM and VRAM

JKrender watches the memory, so Windows never starts swapping and the graphics card does not fill up.



Within 5 % of the limit for 30 s
After their chunks the processes get a fresh Nuke, one at a time (Reload Nuke).

Over the limit
The process started last closes: at once if it still opens the script, else after its chunk.

Coming back
As many come back at once as fit; they start one after another.

Overloaded again
It waits 5 s, then 30, 60 and 150 s when it happens again within 10 minutes.

For a process to come back, one process of the size the processes of this render had (until known: its Cache + 4 GB) must fit under the limit, with a 2 % margin and what the running processes are still going to take. What it waits for, the **State** column in System shows, e.g. *wait: VRAM 75 → 69 %*: the VRAM is at 75 % and must drop under 69 % for one more process to fit. *wait: VRAM 75 → 14 % (3 starting)*: three processes are still starting and JKrender keeps room for them at full size. More states: *wait: 25 s* (a process just fell), *wait: W03 1st frame* (it waits for W03's first frame, to see how much a process takes), *wait: VRAM ok, checking* (there is room; it checks that it stays).

A full card and nothing moves: when the VRAM is over the limit and no process finishes a frame for longer than 4× the usual frame time (at least a minute), Nuke has likely fallen back to the CPU and all processes wait. The process started last then closes at once and its chunk goes back to the queue.

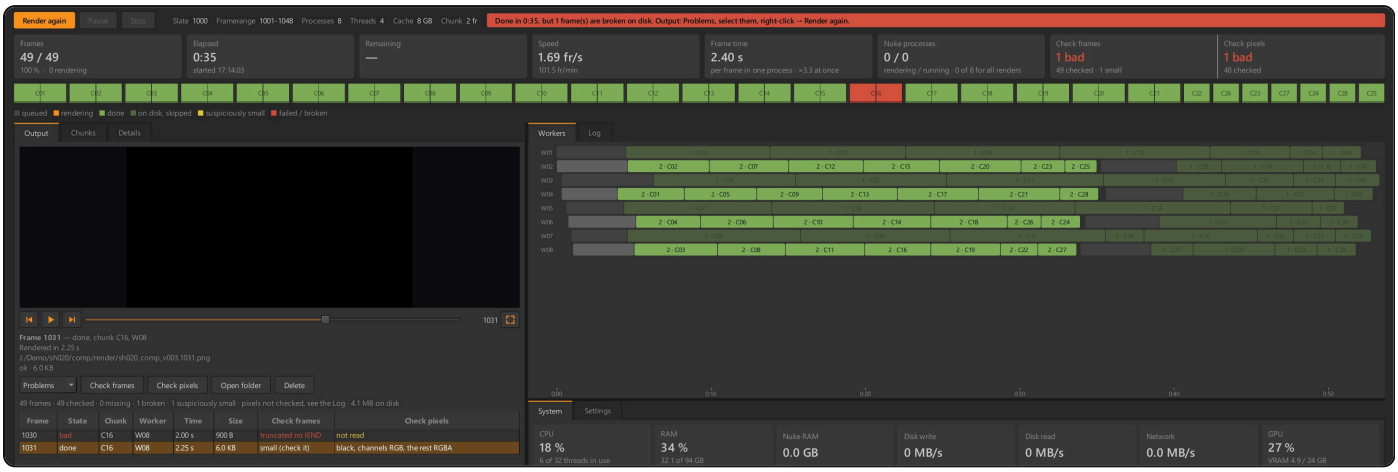
Reload Nuke

While rendering, Nuke keeps memory it no longer needs: a process of a heavy script can grow from 2 to 7 GB of VRAM in its first ~20 frames. A fresh Nuke starts small again, so more processes fit on the card. With the default **When memory is full** this happens only when the RAM or VRAM has stayed within 5 % of its limit **for 30 s without a break** (one reading over it is not enough), and **one process at a time**: the next only once the new Nuke has rendered its first frame and memory is still near the limit. The others go on rendering meanwhile. A process that gets a fresh Nuke shows *busy → reload (VRAM 93 %)* in State; nothing is swapped for the last chunks.

There is no VRAM cap per process. Cache (`-c`) limits only Nuke's RAM cache. The graphics card's memory of one process cannot be capped: Windows cannot do it, and Nuke's Blink cache settings have no effect in a render (measured). So JKrender controls VRAM by the number of processes and by fresh Nukes. Cache is a cache limit, not the whole memory of a process; the real number is in System › RAM.

14 Checks, sounds, crashes and data

Checking the output



A render with a cut-off frame (1030) and a black frame (1031), both found by Check frames and Check pixels by hand: a red bar, red frames in the map, the Problems filter shows only them.

Check	When	What it looks for
Check frames	When the render ends and after Stop (can be turned off), and with the button. Always when a render opens and before Render.	Missing, empty, cut-off and unreadable files (EXR, PNG, JPEG, DPX), suspiciously small frames.
Check pixels	When the render ends, when it opens (can be turned off) and with the button.	EXR, PNG, JPEG, TIFF and DPX: black, empty and one-colour frames, frames the same as another, a different size, channels and bit depth. In EXR also NaN, inf and compression. Other formats: only identical files.
Movie check	For MOV, MP4 and MXF.	Codec, profile, fps, resolution, timecode and frame count against the Write's settings.

Sound after a render

After the render the checks run in the order **Check frames** → **Check pixels**; the sound comes once those that are on are through (with both off, right after the render).

A happy sound: everything is fine. **An error sound**: the render failed, has broken frames or the pixel check found a problem. A render stopped by hand is silent.

When JKrender goes down

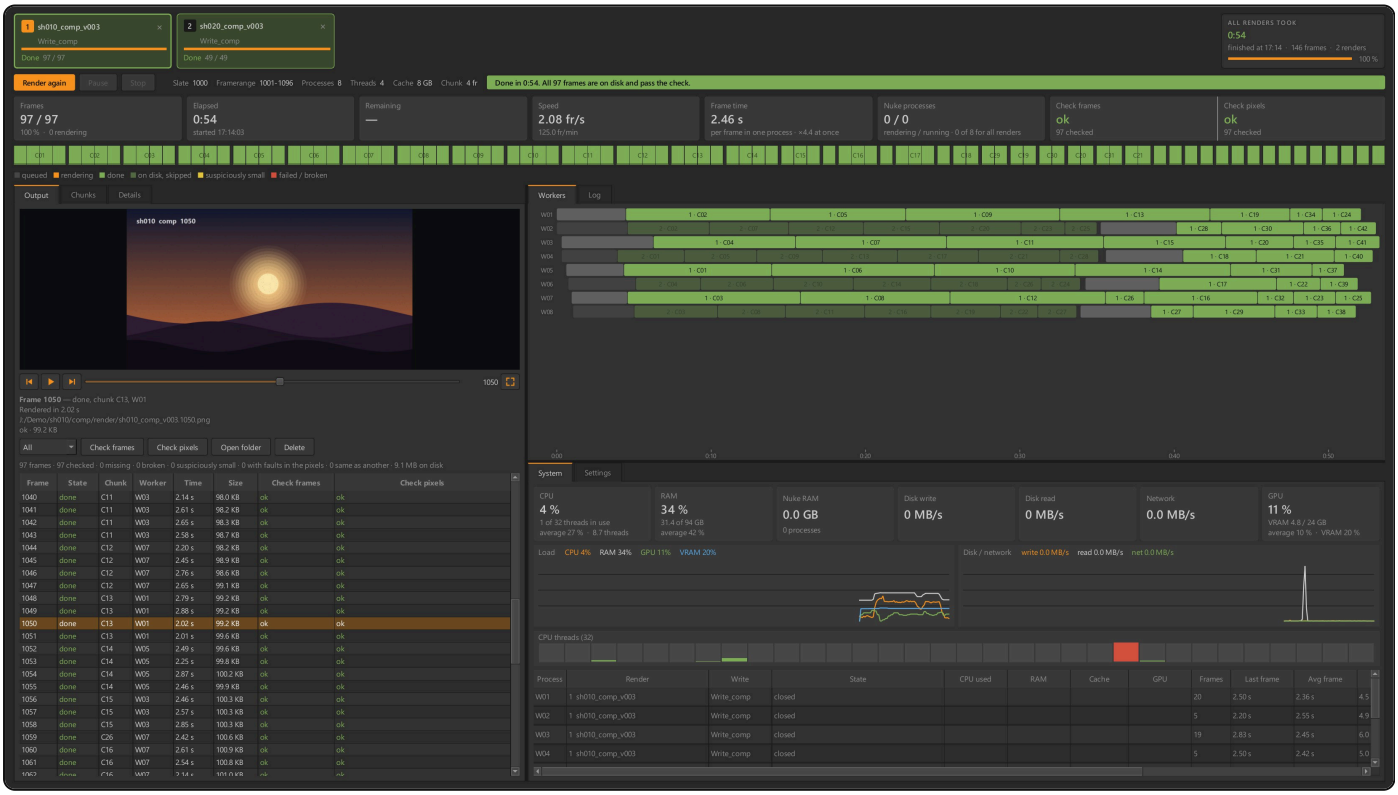
The cards come back the next time JKrender opens (for example with the next render sent from Nuke). What is on disk counts and the Nuke processes left over are ended. A render that was going waits for Render and then renders what is missing. When you close the window normally, nothing comes back.

Render data

Every render writes `report.json` into its log folder: CPU, RAM and graphics card of this PC, settings, speed, memory peaks, closing for limits, Nuke's start, opening the script and the kinds of nodes in the script. **It holds no script names, paths or errors**; a script is known only by an ID that stays the same across its versions. When you run into a bug, send the render's log folder.

15 Mouse and keys: cheat sheet

Where and what	Result
`Ctrl+Alt+R` in Nuke	Sends the selected Writes to JKrender.
Click on a render card	Picks the render; the whole window shows its data.
`Shift` + click on cards	Picks several renders for Render, Render again, Pause, Stop.
Click on a frame in the frame map	Opens the frame in the preview.
`Space`	Play / stop the preview (never stops the render).
Wheel / drag in the preview	Zoom / move the image.
Double click in the preview	The whole frame; when it already is, a big preview.
`Esc`	Back from the big preview.
Right click on a frame (Output)	Render this frame / batch, Delete this frame / batch.
Right click on a chunk (Chunks)	Render again, Show process log, Delete frames from chunk.
Right click on a process (System, Workers)	Pause, Close after this chunk, Close, Remove process, Work on, Add process, Start again, Show process log.
Right click on an empty spot in System	Add process, Remove process.
Click on the System table header	Which columns and in what order.
Dragging a panel border	Resizes the panels (saved).
Framerange + `Enter`	Confirms a new range (Settings › Job).



A finished render: a green bar; the card top right shows how long the renders took.

16 When something does not work

Problem	What to do
Nuke processes take long to start (30–50 s)	Usually Nuke's start (licence, the pipeline's start-up scripts) while the PC renders. The Log splits it: *Nuke started in … / script opened in …*.
Processes close and start again	RAM or VRAM ran out (State in System, Log). Fewer processes with more threads or a smaller Cache; Test this script shows what is fastest.
State says *busy* → *reload*	RAM or VRAM has been near the limit for 30 s; after its chunk the process gets a fresh Nuke (Settings › Reload Nuke). If it happens all the time, more processes run than fit.
The whole render is slower than last time	Check the Cache: too small (e.g. 4 GB for a heavy script) means Nuke throws away and computes again, and frames take several times longer.
The script opens in Nuke with a knob error	E.g. *LensDistortion2.serialiseKnob: Failed to load* (a node from another version, a missing plug-in). JKrender renders on, as Nuke does in the GUI; the message is in the process's log.
Yellow bar "not checked"	The frame check is off (Check frames when a render ends) or did not run. Press Check frames.
Red card or bar	The card's tooltip says what went wrong; Output › the Problems filter shows which frames. Right click *Render this frame / batch*. The bar is worked out again after Check frames and Check pixels by hand too.
No EXR preview	numpy/OpenEXR are missing. Delete `lib/DECLINED` and restart Nuke, or install them as the README says.
The render did not start at once	State Waiting: other renders already use the PC (Max renders at once), or it waits for memory.
Clearing Nuke's cache does not work	After updating JKrender, restart Nuke once.
Not sure what was rendered	The Details panel and Log › Open log folder (script snapshot and job.json).

> **Found a bug?** Send the author the render's log folder (Open Renders Folder) and a description of what happened.